

EX.NO:1.a)

8- QUEENS PROBLEM

CODE:

```
def isSafe(mat, r, c):
    # Check column
    for i in range(r):
        if mat[i][c] == 'Q':
            return False

    # Check upper left diagonal
    i, j = r, c
    while i >= 0 and j >= 0:
        if mat[i][j] == 'Q':
            return False
        i -= 1
        j -= 1

    # Check upper right diagonal
    i, j = r, c
    while i >= 0 and j < len(mat):
        if mat[i][j] == 'Q':
            return False
        i -= 1
        j += 1

    return True

def printSolution(mat):
    for row in mat:
        print(str(row).replace(',', ' ').replace('"', ''))
    print()

def nQueen(mat, r):
    if r == len(mat):
        printSolution(mat)
        return

    for i in range(len(mat)):
        if isSafe(mat, r, i):
            mat[r][i] = 'Q'
            nQueen(mat, r + 1)
            mat[r][i] = '-' # backtrack

if __name__ == "__main__":
    N = int(input("Enter number of queens: "))
    mat = [['-' for _ in range(N)] for _ in range(N)]
    nQueen(mat, 0)
```

OUTPUT:

Enter number of queens: 8

```
[Q - - - - - - -]  
[- - - - Q - - -]  
[- - - - - - - Q]  
[- - - - - Q - -]  
[- - Q - - - - -]  
[- - - - - - Q -]  
[- Q - - - - - -]  
[- - - Q - - - -]
```

```
[Q - - - - - - -]  
[- - - - - Q - -]  
[- - - - - - - Q]  
[- - Q - - - - -]  
[- - - - - - Q -]  
[- - - Q - - - -]  
[- Q - - - - - -]  
[- - - - Q - - -]
```

```
[Q - - - - - - -]  
[- - - - - - Q -]  
[- - - Q - - - -]  
[- - - - - Q - -]  
[- - - - - - Q]  
[- Q - - - - - -]  
[- - - - Q - - -]  
[- - Q - - - - -]
```

```
[Q - - - - - - -]  
[- - - - - - Q -]  
[- - - - Q - - -]  
[- - - - - - Q]  
[- Q - - - - - -]  
[- - - Q - - - -]  
[- - - - - Q - -]  
[- - Q - - - - -]
```

```
[- Q - - - - - -]  
[- - - Q - - - -]  
[- - - - - Q - -]  
[- - - - - - Q]  
[- - Q - - - - -]  
[Q - - - - - - -]  
[- - - - - - Q -]  
[- - - - Q - - -]
```

EX.NO:1.b)

DEPTH FIRST SEARCH

CODE:

```
import networkx as nx

# FUNCTION TO SOLVE DFS
def solveDFS(graph, v, visited):
    visited.add(v)
    print(v, end=' ')

    for neighbour in graph[v]:
        if neighbour not in visited:
            solveDFS(graph, neighbour, visited)

# Create a graph using NetworkX
g = nx.DiGraph()

# Add edges for the graph
g.add_edges_from([
    ('A', 'B'),
    ('A', 'C'),
    ('C', 'G'),
    ('B', 'D'),
    ('B', 'E'),
    ('D', 'F'),
    ('A', 'E')
])

# Visualize the graph
nx.draw(g, with_labels=True)

# Solve DFS for the graph starting from vertex A
print("Following is DFS from (starting from vertex A):")
visited = set()
solveDFS(g, 'A', visited)
|
```

OUTPUT:

```
Following is DFS from (starting from vertex A):
A B D F E C G
```

EX.NO:1.b)**DEPTH FIRST SEARCH – WATER JUG PROBLEM**

```
from collections import deque
def DFS(a, b, target):
    m = {}
    isSolvable = False
    path = []
    q = deque()
    q.append((0, 0))
    while (len(q) > 0):
        u = q.popleft()
        if ((u[0], u[1]) in m):
            continue
        if ((u[0] > a or u[1] > b or u[0] < 0 or u[1] < 0)):
            continue
        path.append([u[0], u[1]])
        m[(u[0], u[1])] = 1
        if (u[0] == target or u[1] == target):
            isSolvable = True
            if (u[0] == target):
                if (u[1] != 0):
                    path.append([u[0], 0])
            else:
                if (u[1] != 0):
                    path.append([0, u[1]])
            sz = len(path)
            for i in range(sz):
                print("(", path[i][0], ",", path[i][1], ")")
            break

        # add all next states
        q.append([u[0], b])
        q.append([a, u[1]])
        for ap in range(max(a, b) + 1):
            c = u[0] + ap
            d = u[1] - ap
            if (c == a or (d == 0 and d >= 0)):
                q.append([c, d])
            c = u[0] - ap
            d = u[1] + ap
            if ((c == 0 and c >= 0) or d == b):
                q.append([c, d])
        q.append([a, 0])
        q.append([0, b])
    if (not isSolvable):
        print("No solution")
Jug1, Jug2, target = 4, 3, 2
print("Path from initial state to solution state ::")
DFS(Jug1, Jug2, target)
```

OUTPUT:

Path from initial state to solution state ::

(0 , 0)
(0 , 3)
(4 , 0)
(4 , 3)
(3 , 0)
(1 , 3)
(3 , 3)
(4 , 2)
(0 , 2)

EXP : MINMAX ALGORITHM

```
from math import inf as infinity
from random import choice
import platform
import time
from os import system

HUMAN = -1
COMP = +1

board = [
    [0, 0, 0],
    [0, 0, 0],
    [0, 0, 0]
]

def evaluate(state):
    if wins(state, COMP):
        score = +1
    elif wins(state, HUMAN):
        score = -1
    else:
        score = 0
    return score

def wins(state, player):
    win_state = [
        [state[0][0], state[0][1], state[0][2]],
        [state[1][0], state[1][1], state[1][2]],
        [state[2][0], state[2][1], state[2][2]],
        [state[0][0], state[1][0], state[2][0]],
        [state[0][1], state[1][1], state[2][1]],
        [state[0][2], state[1][2], state[2][2]],
        [state[0][0], state[1][1], state[2][2]],
        [state[2][0], state[1][1], state[0][2]],
    ]
    if [player, player, player] in win_state:
        return True
    else:
        return False

def game_over(state):
    return wins(state, HUMAN) or wins(state, COMP)

def empty_cells(state):
    cells = []
    for x, row in enumerate(state):
        for y, cell in enumerate(row):
            if cell == 0:
                cells.append([x, y])
    return cells
```

```

def game_over(state):
    return wins(state, HUMAN) or wins(state, COMP)

def empty_cells(state):
    cells = []
    for x, row in enumerate(state):
        for y, cell in enumerate(row):
            if cell == 0:
                cells.append([x, y])
    return cells

main()

```

OUTPUT:

```

Choose X or O
Chosen: X

First to start? [y/n]: Y

Human turn [X]

-----
|  ||  ||  |
-----
|  ||  ||  |
-----
|  ||  ||  |
-----

Use numpad (1..9): 5

Computer turn [O]

-----
|  ||  ||  |
-----
|  || X ||  |
-----
|  || O ||  |
-----

Use numpad (1..9): 1

```

Implement A*

```
from collections import deque
class Graph:
    def __init__(self, adjac_lis):
        self.adjac_lis = adjac_lis
    def get_neighbors(self, v):
        return self.adjac_lis.get(v, [])

    def h(self, n):
        H = {
            'A': 1,
            'B': 1,
            'C': 1,
            'D': 0 # Goal node
        }
        return H.get(n, 1)
    def a_star_algorithm(self, start, stop):
        open_lst = set([start])
        closed_lst = set([])
        poo = {}
        poo[start] = 0
        par = {}
        par[start] = start
        while len(open_lst) > 0:
            n = None
            for v in open_lst:
                if n is None or poo[v] + self.h(v) < poo[n] + self.h(n):
                    n = v
            if n is None:
                print("Path does not exist!")
                return None
            if n == stop:
                reconst_path = []
                while par[n] != n:
                    reconst_path.append(n)
                    n = par[n]
                reconst_path.append(start)
                reconst_path.reverse()
                print("Path found:", reconst_path)
                return reconst_path
            for (m, weight) in self.get_neighbors(n):
                if m not in open_lst and m not in closed_lst:
                    open_lst.add(m)
                    par[m] = n
                    poo[m] = poo[n] + weight
                else:
                    if poo[m] > poo[n] + weight:
                        poo[m] = poo[n] + weight
                        par[m] = n
                        if m in closed_lst:
                            closed_lst.remove(m)
                            open_lst.add(m)
                    open_lst.remove(n)
                    closed_lst.add(n)
            print("Path does not exist!")
            return None
adjac_lis = {
    'A': [('B', 1), ('C', 3), ('D', 7)],
    'B': [('D', 5)],
    'C': [('D', 12)]
}
graph1 = Graph(adjac_lis)
graph1.a_star_algorithm('A', 'D')
```


OUTPUT:

```
----- RESTART: C:\JSE
==
Path found: ['A', 'B', 'D']
>>> |
```

AO* SEARCH:

```
class AOSTar:

    def __init__(self, heuristics, conditions, weight=1):

        self.H = heuristics
        self.conditions = conditions
        self.weight = weight

        self.parent = {}

        # 0 = unvisited, 1 = solved
        self.status = {}

        self.solution_graph = {}

    def get_node_status(self, node):
        return self.status.get(node, 0)

    def set_node_status(self, node, status):
        self.status[node] = status

    def get_heuristics(self, node):
        return self.H.get(node, 0)

    def compute_minimum_cost_child_nodes(self, v):

        minimum_cost = 0

        cost_to_child_nodes_dict = {}

        cost_to_child_nodes_dict['AND'] = []
        cost_to_child_nodes_dict['OR'] = []

        if v in self.conditions:

            cond = self.conditions[v]

            # AND condition
            if 'AND' in cond:

                and_nodes = cond['AND']

                and_cost = sum(
                    self.get_heuristics(node) + self.weight
                    for node in and_nodes
                )

                cost_to_child_nodes_dict['AND'] = (
                    and_cost,
                    and_nodes
                )

            # OR condition
            if 'OR' in cond:
```

```

        or_nodes = cond['OR']

        for node in or_nodes:

            or_cost = self.get_heuristics(node) + self.weight

            cost_to_child_nodes_dict['OR'].append(
                (or_cost, [node])
            )

        return cost_to_child_nodes_dict
def ao_star(self, v, back_tracking):
    print(f"EXPANDING NODE: {v}")
    if self.get_node_status(v) >= 0:

        minimum_cost_dict = \
            self.compute_minimum_cost_child_nodes(v)

        best_path_type = None

        min_cost = float('inf')

        best_nodes = []

        # Check AND path
        if minimum_cost_dict['AND']:

            min_cost, best_nodes = \
                minimum_cost_dict['AND']

            best_path_type = 'AND'

        # Check OR paths
        for or_cost, or_nodes in \
            minimum_cost_dict['OR']:

            if or_cost < min_cost:

                min_cost = or_cost

                best_nodes = or_nodes

                best_path_type = 'OR'

        if best_path_type:

            self.H[v] = min_cost

            self.solution_graph[v] = best_nodes

            solved = True

```

```

        for child in best_nodes:

            self.parent[child] = v

            if self.get_node_status(child) != 1:
                solved = False

        if solved:
            self.set_node_status(v, 1)

        # Backtracking
        if v != root_node:
            self.ao_star(self.parent[v], True)

        # Expand child nodes
        if not back_tracking:

            for child in best_nodes:

                self.set_node_status(child, 0)

                self.ao_star(child, False)

h = {
    'A': 1,
    'B': 5,
    'C': 2,
    'D': 4,
    'E': 7,
    'F': 9,
    'G': 3,
    'H': 0,
    'I': 0,
    'J': 0
}
conditions = {
    'A': {
        'OR': ['B'],
        'AND': ['C', 'D']
    },
    'B': {
        'OR': ['E', 'F']
    },
    'C': {
        'OR': ['G'],
        'AND': ['H', 'I']
    },
    'D': {
        'OR': ['J']
    }
}
root_node = 'A'

```

OUTPUT:

```

EXPANDING NODE: A
EXPANDING NODE: B
EXPANDING NODE: A
EXPANDING NODE: E

```

```

Final Heuristics: {'A': 8, 'B': 8, 'C': 2, 'D': 4, 'E': 7, 'F': 9, 'G': 3, 'H': 0, 'I': 0, 'J': 0}
Solution Graph: {'A': ['C', 'D'], 'B': ['E']}

```

Implementation of Unification and Resolution Algorithm.

```
def get_var(x):
    """Checks if a term is a variable (defined here as a lowercase string)."""
    return isinstance(x, str) and x.islower()

def unify(x, y, theta={}):
    """
    Main Unification Function.
    x, y: Terms (Constants, Variables, or Lists)
    theta: The substitution dictionary (bindings)
    """

    if theta is None:
        return None

    elif x == y:
        return theta

    elif get_var(x):
        return unify_var(x, y, theta)

    elif get_var(y):
        return unify_var(y, x, theta)

    elif isinstance(x, list) and isinstance(y, list):

        if len(x) != len(y):
            return None

        # Unify the first elements
        new_theta = unify(x[0], y[0], theta)

        # Unify remaining elements
        return unify(x[1:], y[1:], new_theta)

    else:
        return None

def unify_var(var, val, theta):
    """Handles variable binding and checks for existing substitutions."""

    if var in theta:
        return unify(theta[var], val, theta)

    elif get_var(val) and val in theta:
        return unify(var, theta[val], theta)

    elif occur_check(var, val, theta):
        return None

    else:
        # Add new substitution
        return {**theta, var: val}
```

```

def occur_check(var, val, theta):
    """Safety check to avoid infinite substitutions."""

    if var == val:
        return True

    elif get_var(val) and val in theta:
        return occur_check(var, theta[val], theta)

    elif isinstance(val, list):
        return any(occur_check(var, v, theta) for v in val)

    return False

# --- Test Cases ---

print("1. Simple Variable:", unify('x', 'A'))

print("2. Matching Lists:",
      unify(['P', 'x', 'y'], ['P', 'A', 'B']))

print("3. Recursive Bind:",
      unify(['f', 'x'], ['f', ['g', 'y']]))

print("4. Mismatch:",
      unify(['P', 'A'], ['Q', 'A']))

print("5. Occur Check:",
      unify('x', ['f', 'x']))

```

OUTPUT:

```

1. Simple Variable: {'x': 'A'}
2. Matching Lists: {'x': 'A', 'y': 'B'}
3. Recursive Bind: {'x': ['g', 'y']}
4. Mismatch: None
5. Occur Check: None

```

RESOLUTION PROGRAM

CODE:

```
import copy

# ----- Unification Functions -----

def get_var(x):
    return isinstance(x, str) and x.islower()

def unify(x, y, theta={}):
    if theta is None:
        return None

    elif x == y:
        return theta

    elif get_var(x):
        return unify_var(x, y, theta)

    elif get_var(y):
        return unify_var(y, x, theta)

    elif isinstance(x, list) and isinstance(y, list):
        if len(x) != len(y):
            return None

        new_theta = unify(x[0], y[0], theta)

        return unify(x[1:], y[1:], new_theta)

    return None

def unify_var(var, val, theta):
    if var in theta:
        return unify(theta[var], val, theta)

    elif get_var(val) and val in theta:
        return unify(var, theta[val], theta)

    elif occur_check(var, val, theta):
        return None

    else:
        return {**theta, var: val}
```



```

# ----- Main Resolution Algorithm -----

def main_resolution(kb, goal):

    # Negate the goal
    clauses = copy.deepcopy(kb)
    clauses.append(['not', goal])

    print("Initial Clauses:")
    print(clauses)

    i = 0

    while i < len(clauses):

        for j in range(i + 1, len(clauses)):

            resolvents = resolve(clauses[i], clauses[j])

            for res in resolvents:

                # Empty clause found
                if res == []:
                    print("\nResolved", clauses[i],
                        "and", clauses[j], "to get []")

                    return True

                if res not in clauses:
                    clauses.append(res)

                    print("New clause derived:", res)

            i += 1

    return False

# ----- Knowledge Base -----

# 1. Human(x) -> Mortal(x)
# 2. Human(Socrates)

kb = [
    [['not', ['Human', 'x']], ['Mortal', 'x']],
    [['Human', 'Socrates']]
]
goal = ['Mortal', 'Socrates']

# ----- Run Program -----

if main_resolution(kb, goal):
    print("\nRESULT: Goal is PROVEN (Contradiction found!)")
else:
    print("\nRESULT: Goal could not be proven.")

```

OUTPUT:

```
-----
Initial Clauses:
[[['not', ['Human', 'x']], ['Mortal', 'x']], [['Human', 'Socrates']], [['not', ['Mortal', 'Socrates']]]]

RESULT: Goal could not be proven.
```

FORWARD CHAINING:

```
# Forward Chaining Implementation

# Rules: IF condition THEN result
rules = [
    (["A", "B"], "C"),
    (["C"], "D"),
    (["D"], "E")
]

facts = ["A", "B"] # Initial facts
goal = "E"

def forward_chaining(rules, facts, goal):
    inferred = set(facts)

    while True:
        new_inferred = False

        for conditions, result in rules:
            if all(cond in inferred for cond in conditions):
                if result not in inferred:
                    print(f"Inferred: {result}")
                    inferred.add(result)
                    new_inferred = True

                if result == goal:
                    return True

        if not new_inferred:
            return False

# Run
if forward_chaining(rules, facts, goal):
    print("Goal reached!")
else:
    print("Goal not reachable.")
```

OUTPUT:

```
=====
=
Inferred: C
Inferred: D
Inferred: E
Goal reached!
```

BACKWARD REASONING:

CODE:

```
# Backward Chaining Implementation

# Knowledge base: rules (IF conditions THEN conclusion)
rules = [
    (["A", "B"], "C"),    # A AND B -> C
    (["D"], "A"),          # D -> A
]

# Known facts
facts = {"B", "D"}

def backward_chain(goal, facts, rules, visited=None):
    if visited is None:
        visited = set()

    print(f"Trying to prove: {goal}")

    # If goal is already a known fact
    if goal in facts:
        print(f"{goal} is a known fact.")
        return True

    # Avoid infinite loops
    if goal in visited:
        return False
    visited.add(goal)

    # Check rules that conclude the goal
    for conditions, conclusion in rules:
        if conclusion == goal:
            print(f"Checking rule: {conditions} -> {conclusion}")

            # Try to prove all conditions
            if all(backward_chain(cond, facts, rules, visited) for cond in conditions):
                print(f"{goal} is proven.")
                return True

    print(f"Failed to prove: {goal}")
    return False

# Test the backward chaining
goal = "C"
result = backward_chain(goal, facts, rules)

print("\nFinal Result:", result)
```

OUTPUT:

```
=====
=
Trying to prove: C
Checking rule: ['A', 'B'] -> C
Trying to prove: A
Checking rule: ['D'] -> A
Trying to prove: D
D is a known fact.
A is proven.
Trying to prove: B
B is a known fact.
C is proven.

Final Result: True
```

FUZZY:

```
import numpy as np
import skfuzzy as fuzz
from skfuzzy import control as ctrl

# Define fuzzy variables
temperature = ctrl.Antecedent(np.arange(0, 41, 1), 'temperature')
fan_speed = ctrl.Consequent(np.arange(0, 101, 1), 'fan_speed')

# Membership functions for temperature
temperature['cold'] = fuzz.trimf(temperature.universe, [0, 0, 20])
temperature['warm'] = fuzz.trimf(temperature.universe, [10, 20, 30])
temperature['hot'] = fuzz.trimf(temperature.universe, [20, 40, 40])

# Membership functions for fan speed
fan_speed['low'] = fuzz.trimf(fan_speed.universe, [0, 0, 50])
fan_speed['medium'] = fuzz.trimf(fan_speed.universe, [25, 50, 75])
fan_speed['high'] = fuzz.trimf(fan_speed.universe, [50, 100, 100])

# Define rules
rule1 = ctrl.Rule(temperature['cold'], fan_speed['low'])
rule2 = ctrl.Rule(temperature['warm'], fan_speed['medium'])
rule3 = ctrl.Rule(temperature['hot'], fan_speed['high'])

# Control system
fan_ctrl = ctrl.ControlSystem([rule1, rule2, rule3])
fan_sim = ctrl.ControlSystemSimulation(fan_ctrl)

# Input value
fan_sim.input['temperature'] = 25

# Compute output
fan_sim.compute()

# Output result
print("Fan Speed:", fan_sim.output['fan_speed'])

# Visualize (optional)
temperature.view()
fan_speed.view()
```

OUTPUT:

```
===== RESTART: C:/Python/fuzzy_logic.py =====

Fan Speed: 50.0

>>>
```

BLOCK WORLD

```
class BlocksWorld:

    def __init__(self, initial, goal):
        self.initial = initial
        self.goal = goal

    def is_goal(self, state):
        return state == self.goal

    def get_top_blocks(self, state):
        """Return blocks that have nothing on top of them"""

        tops = set(state.keys()) - set(state.values())
        return list(tops)

    def move(self, state, block, destination):
        """Move a block to a new destination"""

        new_state = state.copy()
        new_state[block] = destination

        return new_state

    def get_possible_moves(self, state):

        moves = []

        top_blocks = self.get_top_blocks(state)

        for block in top_blocks:

            # Move to table
            if state[block] != "table":
                moves.append((block, "table"))

            # Move onto another top block
            for dest in top_blocks:

                if dest != block:
                    moves.append((block, dest))

        return moves

    def solve(self):

        from collections import deque

        visited = set()

        queue = deque()

        queue.append((self.initial, []))

        while queue:
```

```

        state, path = queue.popleft()

        if self.is_goal(state):
            return path

        state_tuple = tuple(sorted(state.items()))

        if state_tuple in visited:
            continue

        visited.add(state_tuple)

        for move in self.get_possible_moves(state):

            new_state = self.move(state, move[0], move[1])

            queue.append((new_state, path + [move]))

    return None

# ----- Example -----

initial_state = {
    'A': 'table',
    'B': 'A',
    'C': 'table'
}

goal_state = {
    'A': 'B',
    'B': 'C',
    'C': 'table'
}

bw = BlocksWorld(initial_state, goal_state)

solution = bw.solve()

print("Solution steps:")

if solution:
    for step in solution:
        print(f"Move {step[0]} to {step[1]}")
else:
    print("No solution found")

```

OUTPUT:

```

===== RESTART: C:/Python/blocks_world.py =====

Solution steps:
Move B to table
Move A to B
Move B to C
>>>

```